Module 11: Vibe Coding for Product Managers

Detailed supplement: the first prompt, iterating in small steps, debugging, connecting data, and a project you can show in an interview



Written by Varun Kumar, for SkillNavora · Draft

Official site: www.skillnavora.in

Prepared for Skillnavora — Learn. Practise. Implement. Grow.

Coverage map: what is new and what is already in the module

The main Module 11 handbook covers vibe coding, prompts and the PM use cases in depth. This supplement adds the deck's working rules: the first prompt as a build brief, one feature per prompt, starting over when the first result is far off, a debugging mindset, data with a backend, and the interview project.

Topic (Week 11 deck)	Handbook section	Status here
Vibe coding versus traditional coding	11.x	Extended: a factor table
The first prompt as a build brief	11.x (partial)	New: a template with each part explained
Rule of three: one feature per prompt	—	New: why and how to iterate in small steps
Start over when the result is far off	—	New: the 60 to 70 percent rule
Live mini-app: idea to prompt to app	11.x	Extended: the calculator and tracker examples
Debugging mindset and common fixes	11.x (partial)	New: a problem, cause and fix table
Best practices for reliable AI apps	—	New: versioned prompts and acceptance criteria
Data and a backend (Supabase)	—	New: frontend shows, backend stores
PM use cases and the project	11.x (partial)	Extended: three project ideas with deliverables
Showing AI-building skills in interviews	—	New: what interviewers look for

NOTE ON THE SOURCE

Several slides are screenshots with no readable text. Tool names and features change quickly; check each tool's current documentation before you build.

EXTENDED · WHAT IT IS

Vibe coding versus traditional coding

Vibe coding means building an app by describing what you want in plain language, then refining it through conversation. Traditional coding writes and compiles syntax. Vibe coding describes, tests and refines. It is built for speed and exploration, not for every precise engineering task.

Factor	Traditional coding	Vibe coding
Language	Syntax-based	Natural language
Workflow	Write, compile, debug	Describe, test, refine
Users	Engineers	Builders and PMs
Goal	Precision	Speed and exploration

NEW · THE FIRST PROMPT

Write the first prompt as a build brief

The first prompt matters most. It sets the direction for everything after it. Think of it as the brief you would give an engineer, with a one-line overview, the core use case and the main screens.

FIRST PROMPT TEMPLATE

```
I need a [type of application] with [list features]. It will be used by [target audience] to solve [core value]. It will be successful if the user can [achieve X]. Start with the main page containing: [page requirements]. Use a clean, responsive design.
```

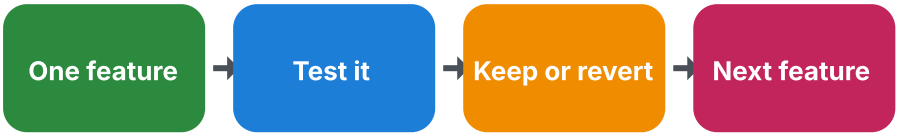
RULE

If the first result is not even 60 to 70 percent of what you had in mind, start over with a better brief and screenshots of similar products.

NEW · ITERATE IN SMALL STEPS

One to three things per prompt

AI cannot do too many things at once, and asking for everything tends to create bugs. Give it at most three things per prompt. Build an MVP, then iterate on each version. When you are unsure, revert to the previous version.



The small-step loop for vibe coding

NEW · DEBUGGING

A debugging mindset and common fixes

AI mistakes are usually communication gaps. Fix one issue at a time, ask the AI to explain its code, and iterate like a conversation.

Problem	Usual cause	Fix
Code error	Missing context	Add step-by-step logic
Wrong format	Vague instruction	Specify the exact format
Unexpected output	Overloaded prompt	Break it into sub-tasks
Slow performance	Too many dependencies	Simplify the logic flow

Best practices: save each prompt version; keep each prompt to one clear outcome; add acceptance criteria (for example, 'the total must show the correct count'); test with sample inputs after each major change.

NEW · DATA AND BACKENDS

Give your app a memory

The frontend shows data; the backend stores it. A managed database such as Supabase can give the app a memory for notes, feedback or tasks, without you writing SQL. API connectors fetch outside data (for example, weather) and save results.



How a vibe-coded app gets a memory

PROJECT

Build your own tool

Choose one idea and build it as a working demo, with the prompt documentation that shows how you got there.

Project idea	What it does	Deliverable
Feature prioritisation dashboard	Ranks features from a sheet of scores	Working demo and prompt log
Team task notifier	Sends Slack alerts from a stored task list	Demo and a list of the prompts used
Product feedback summariser	Summarises feedback from a sheet	Demo and a short debugging note

INTERVIEW

Show your AI-building skills

Interviewers look for three things: clear problem framing that leads to a good prompt; debugging logic that shows how you iterated; and product thinking that shows impact and usability.

Quiz with answers

1. What is the 60 to 70 percent rule?
   Answer: if the first result is far from what you had in mind, start over with a better brief.
2. How many things should one prompt ask for?
   Answer: at most three.
3. What is the frontend's job in an app with a backend?
   Answer: to show data; the backend stores and manages it.
4. What do AI mistakes usually reveal?
   Answer: a communication gap, so fix one issue at a time.
5. What should an acceptance criterion look like?
   Answer: a testable statement, such as 'the total must show the correct count'.

MODULE 11 · FINAL

Completion checklist

Completion checklist

Done	Item
☐	Read the coverage map
☐	Wrote a first prompt as a build brief
☐	Built one feature in small steps, with a revert
☐	Fixed one bug using the debugging table
☐	Connected one piece of data through a backend
☐	Shared the project with prompt documentation